Project: Iris Classification

Dataset Used: Iris.csv

This project uses the Iris dataset to classify flower species based on features like sepal and petal dimensions. Multiple machine learning algorithms are applied and their performance is evaluated.

Project Goal: Train a module to predict the classes using Iris.csv dataset.

Technologies Used:

- **NumPy** : Numerical Operations
- **Pandas** : Data Handling
- **Scikit-Learn** : ML models & tools

Project Coverage – Iris Classification:

Category	Skill Covered
Data preprocessing	Handling Dataset using Pandas
Create Learning Dataset	Select the required columns for passing as Input.
Training the module	Algorithms to train the module using the dataset.
Predict the result	Algorithms to predict the species class

Project covers data preprocessing, multiple ML models, prediction, and evaluation for iris classification. we are using different classes from the sklearn module to predict the species class with different algorithms as shown below.

1.Algorithm: Support Vector Machine (SVM):

- Draws a line/plane between classes
- Chooses the boundary with maximum distance from nearest points

SVM is a supervised learning algorithm that classifies data by finding the optimal boundary with maximum margin between classes.

```
from sklearn.svm import SVC
classifier = SVC()
print(classifier)

# Train model
classifier.fit(X_train, y_train)

# Predict test data
y_pred = classifier.predict(X_test)

# Evaluate accuracy
from sklearn.metrics import accuracy_score
print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

In this algorithm we are importing SVC class from svm module and training the model with the training set. After that we are predicting the accuracy and printing it.

2.Logistic Regression:

- Uses a sigmoid function to convert output into probability (0 to 1)
- Final class is decided using a threshold (usually 0.5)

```
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression()
print(classifier)

classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

Logistic Regression is a supervised learning algorithm used for classification that predicts the probability of a class. In this algorithm we are importing LogisticRegression classs from linear_model sub-module and using fit function to train the model. After that predicting the accuracy and printing it.

3.Naive Bayes:

- Calculates probability of each class
- Chooses class with highest probability
- Assumes features are independent (naive assumption)

```
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
print(classifier)

classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

Naive Bayes is a classification algorithm based on Bayes' theorem that assumes all features are independent.

4. Decision Tree Classifier:

- Data is divided based on feature values
- Forms a tree structure:
- Root → Decision nodes → Leaf nodes
- Final leaf node gives the class label

```
from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier()
print(classifier)

classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

Decision Tree Classifier is a model that classifies data by splitting it into branches using decision rules sub-module and using fit function to train the model. After that predicting the accuracy and printing it.

5. Random Forest Classifier:

- Creates multiple random subsets of data
- Builds a decision tree for each subset
- Combines predictions from all trees
- Outputs the most common class

```
from sklearn.ensemble import RandomForestClassifier
classifier = RandomForestClassifier()
print(classifier)

classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

Random Forest Classifier is an ensemble algorithm that uses multiple decision trees and predicts the final class using majority voting.

What you have learned:

In This project we have learned in the following.

- Learned how to load and manage data using Pandas
- Understood how to select features (X) and target (y)
- Understood importance of feature scaling using Scikit-learn
- Difference between training and testing
- Learned how to measure performance using accuracy

Conclusion:

The project concludes that machine learning models can effectively classify iris species, and performance improves by comparing multiple algorithms. Multiple algorithms (SVM, Logistic Regression, Naive Bayes, Decision Tree, Random Forest) were appliedModels were trained, tested, and evaluated using accuracy score